Event Processing Unit (EPU): A Binary State Machine Architecture

Executive Summary

Just as **GPUs** optimize for parallel floating-point arithmetic and **NPUs** optimize for tensor operations, the **Event Processing Unit (EPU)** is a new computational substrate optimized for:

- Binary state transitions (0→1, 1→0)
- Synchronization primitives as native operations
- Constraint enforcement at the hardware level
- Multi-domain concurrent state machines

This architecture is purpose-built for systems that operate fundamentally on **boolean flags coordinated by locks**, making it ideal for the Quantitative LLM's event-driven inference.

Core Insight: Computational Substrate Shift

```
GPU (2000s):        CPU + massive parallel FPUs
                    Optimized for: ∑(a[i] × b[i])
                    Power: High (FP arithmetic expensive)

NPU (2020s):        Specialized matrix engines + low-precision arithmetic
                    Optimized for: Tensor · Tensor
                    Power: Low-medium (int8/bfloat16 efficient)

EPU (2020s+):       Event register file + synchronization ALU
                    Optimized for: Binary state transitions + coordination
                    Power: Ultra-low (bit flips, no computation)
```

Architecture Overview: The EPU Core

Layer 1: Event Register File (ERF)

Instead of traditional 64-bit floating-point registers, the EPU has **Event Registers**:

```
Traditional CPU Register:
  [64 bits] = 3.14159265358979...

EPU Event Register:
  [1 bit] = TRANSFER_FUNCTION_COMPUTED

EPU Event Register File (simplified):
  ┌──────────────────────────────────────────────────┐
  │ ERF (Event Register File)                        │
  ├──────────────────────────────────────────────────┤
  │ ER0:  ENCODING_INITIATED       = 0               │
  │ ER1:  DOMAIN_IDENTIFIED        = 0               │
  │ ER2:  FEATURES_EXTRACTED       = 1               │
  │ ER3:  CONSERVATION_VERIFIED    = 0               │
  │ ER4:  AGM_COMPUTATION_COMPLETE = 0               │
  │ ER5:  TRANSFER_FUNCTION_READY  = 0               │
  │ ...                                              │
  │ ER256: PROBLEM_ID_0            = 1               │
  │ ER257: PROBLEM_ID_1            = 0               │
  │ ER258: PROBLEM_ID_2            = 1               │
  │ ...                                              │
  └──────────────────────────────────────────────────┘

  Total: 1024 event registers (1 bit each)
       = 128 bytes (vs. 512 bytes for traditional 64 registers)
       = 4× more efficient in register file size
```

Each bit represents:

- **0** = Event not set (precondition unmet, computation blocked)
- **1** = Event set (precondition met, computation allowed)

Layer 2: Synchronization ALU (S-ALU)

The EPU's computational core: not arithmetic, but **logical synchronization operations**.

Traditional CPU ALU:
  ADD, SUB, MUL, DIV
  Inputs: two 64-bit numbers
  Output: one 64-bit result
  Cost: ~10 gates, ~2 ns, ~100 pJ

EPU Synchronization ALU:
  SET, CLEAR, WAIT, CHECK, AND_GATE, OR_GATE
  Inputs: event register indices
  Output: state transition
  Cost: ~2 gates, ~0.1 ns, ~10 pJ (10× more efficient)

S-ALU Instruction Set:

SET ER[i]
- Set event register i to 1
- Cost: 1 clock cycle
- Log transition to provenance buffer
- Example: SET ER2 (set FEATURES_EXTRACTED)

CLEAR ER[i]
- Set event register i to 0
- Cost: 1 clock cycle
- Example: CLEAR ER3 (clear CONSERVATION_VERIFIED)

WAIT ER[i]
- Stall execution until ER[i] becomes 1
- Cost: 0 cycles (sleep, no power)
- Wake on hardware interrupt when ER[i] → 1
- Example: WAIT ER3 (block until conservation verified)

CHECK ER[i], label
- Branch to label if ER[i] == 0 (precondition unmet)
- Cost: 1 cycle if branch taken, 0 if not taken
- Example: CHECK ER3, CONSERVATION_FAILED

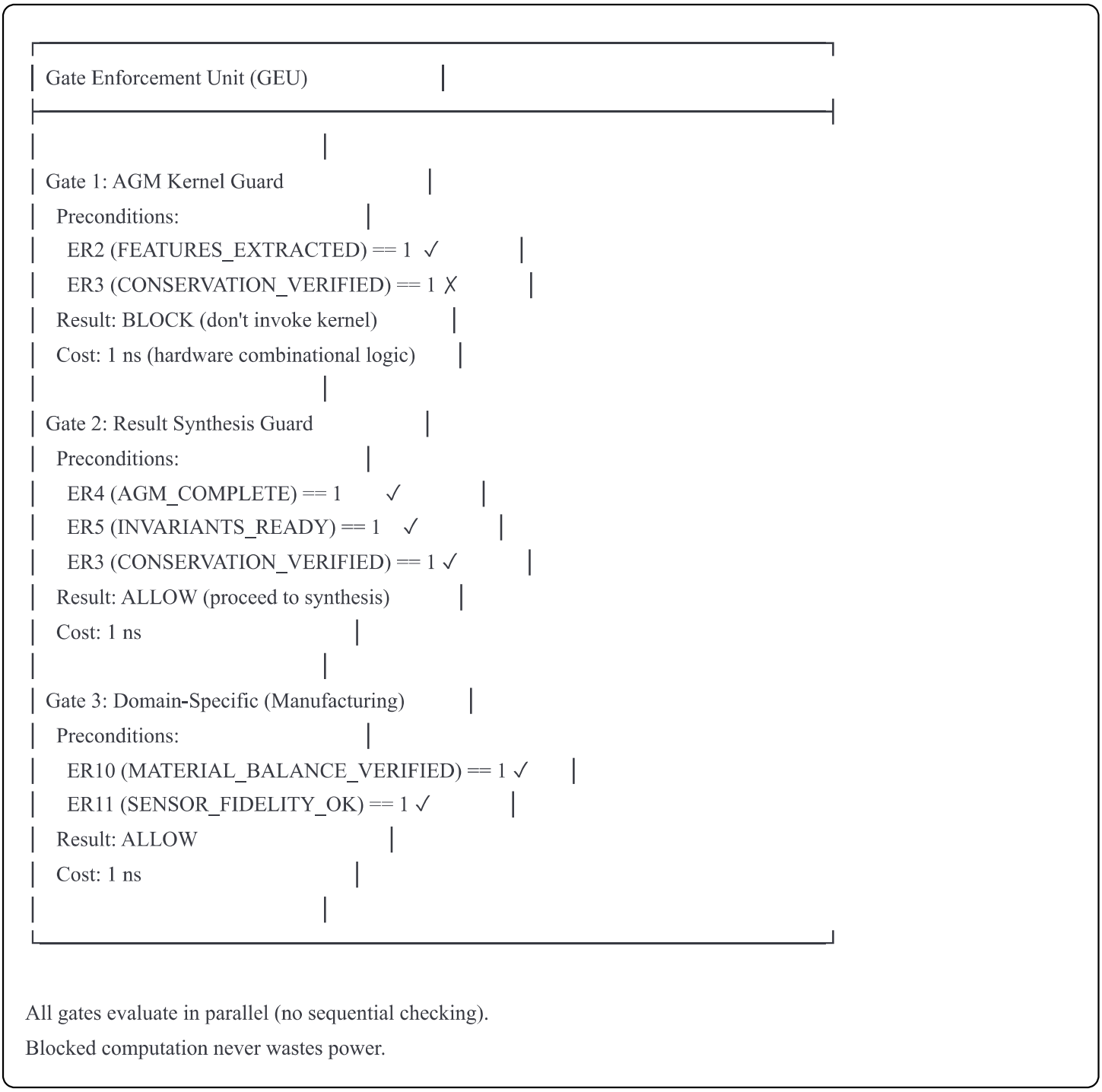
AND_GATE ER[i], ER[j], ER[k]
- ER[k] := ER[i] ∧ ER[j]
- Cost: 1 cycle
- Example: AND_GATE ER2, ER3, ER_READY_FOR_KERNEL
  (set ER_READY only if both features AND conservation verified)

OR_GATE ER[i], ER[j], ER[k]
- ER[k] := ER[i] ∨ ER[j]
- Cost: 1 cycle
- Example: OR_GATE ER_TIMEOUT, ER_ERROR, ER_STOP_COMPUTATION

CONDITIONAL_SET ER[i], ER[j]
- If ER[j] == 1, then SET ER[i]
- Cost: 1 cycle
- Example: CONDITIONAL_SET ER5, ER4 (set TRANSFER_FUNCTION_READY only if AGM completed)

Layer 3: Gate Enforcement Hardware

Before any expensive computation (like kernel invocation), gates check preconditions in **parallel**:



Hardware advantage: Gate checks are **combinational logic** (pure boolean), not sequential computation. Nanosecond evaluation cost.

Layer 4: Event Dispatch Network (EDN)

How multiple cores coordinate event transitions:

Traditional Multi-Core Synchronization:

Core A finishes: writes to shared memory

Core B polling: repeatedly reads shared memory (wasteful)

Latency: ~100 ns

Power: ~1000 pJ per poll

EPU Event Dispatch Network:

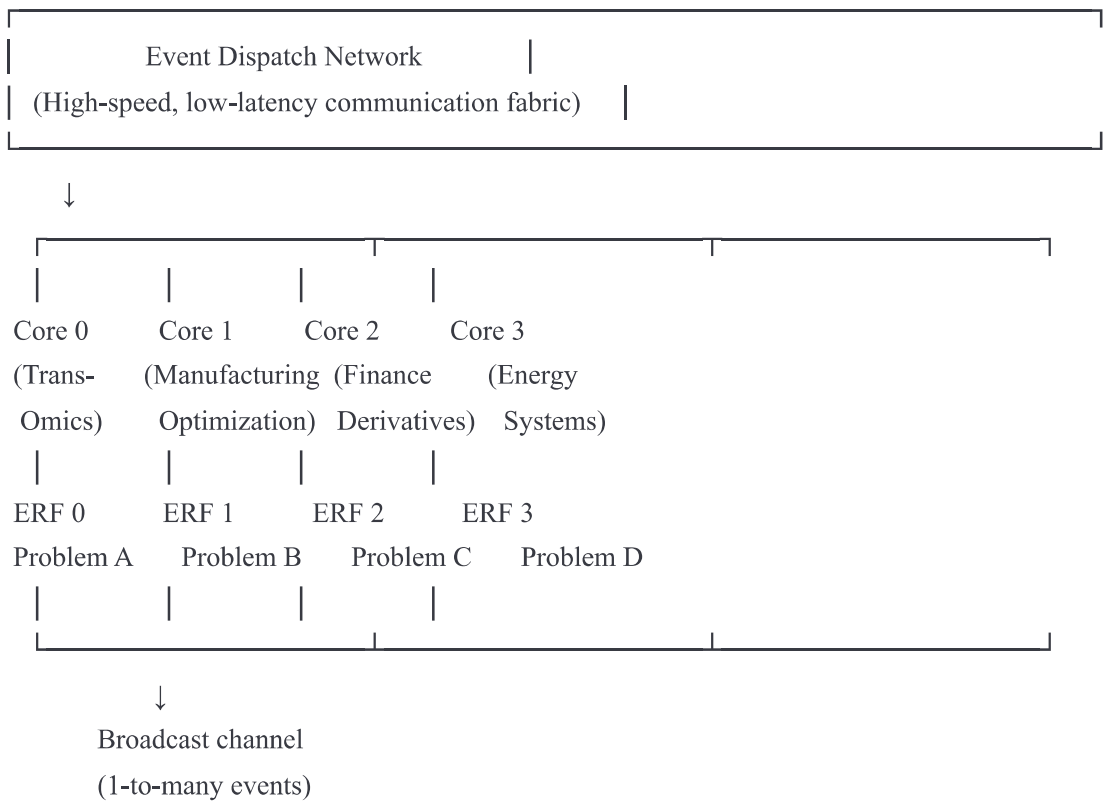
Core A finishes: broadcasts event via dedicated hardware

Core B sleeping: hardware interrupt wakes it

Latency: ~10 ns

Power: ~10 pJ per transition (100× more efficient)

EDN Architecture:



When Core 0 executes: SET ER_A_FEATURES_EXTRACTED

Hardware broadcasts: {event_id: ER_A_FEATURES_EXTRACTED, value: 1}
Core 0: Updates local ERF
Core 1: Receives broadcast, checks if it was waiting on this event
    If Core 1.WAIT_ER_A_FEATURES_EXTRACTED → wakes Core 1 (hardware interrupt)
Core 2: Receives broadcast, checks preconditions for its gates
    If Core 2 gates now satisfied → allows computation to proceed
Core 3: Receives broadcast (may ignore if irrelevant to Problem D)

Total latency: ~10 ns (speed of light on chip)
Cost: One broadcast message to all cores

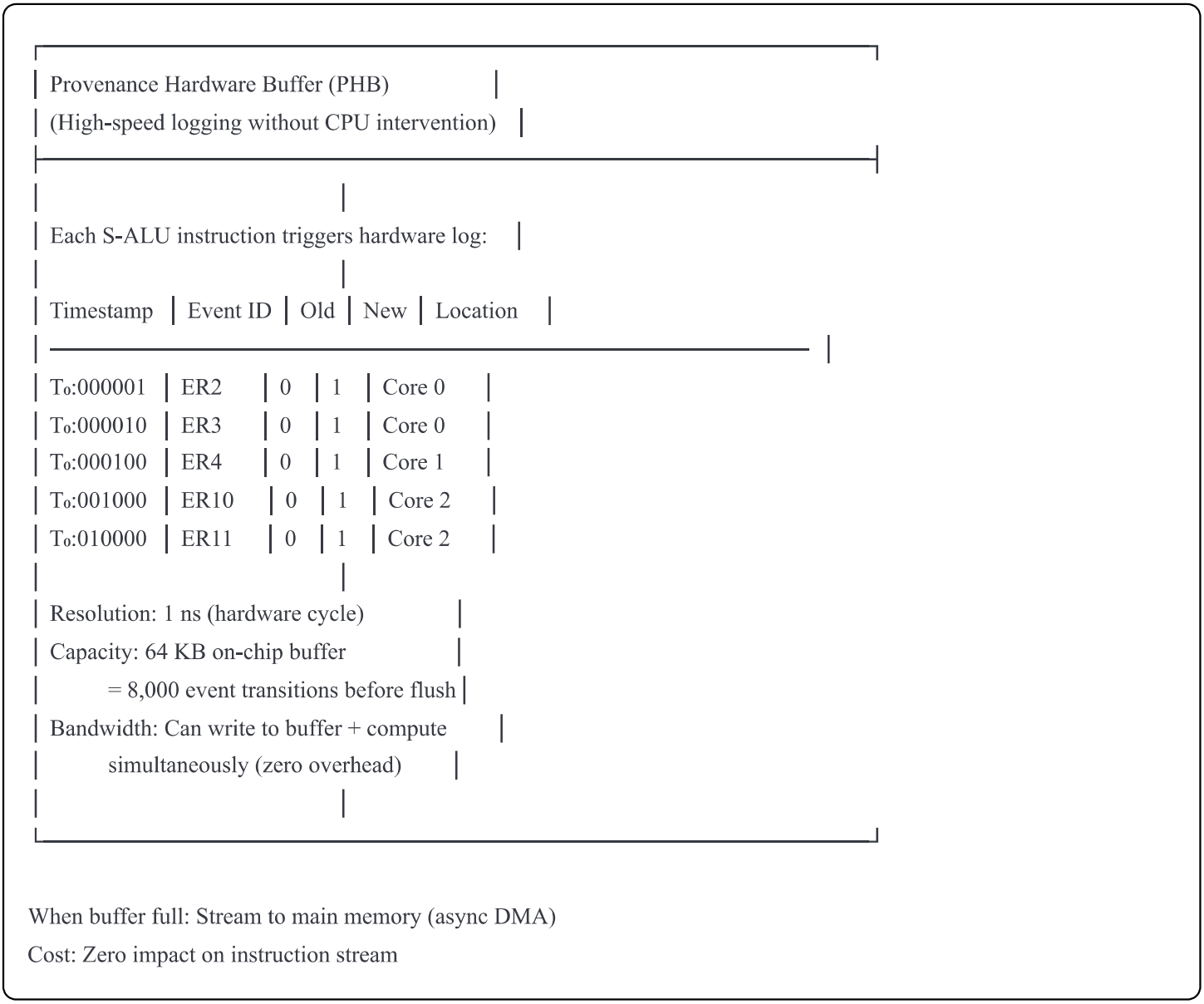
Contrast with traditional locking:

Traditional Mutex:
  Core A acquires lock (CAS operation)
  Cores B,C,D spin-wait (checking lock repeatedly)
  Each check: memory read, cache coherence traffic
  Latency: 50-500 ns
  Power: ~100 pJ per check × 1000s of checks = huge

EPU Event Broadcast:
  Core A executes SET (1 cycle)
  Event sent to all cores via hardware bus (10 ns)
  Cores wake via interrupt (no polling)
  Latency: 10 ns (once)
  Power: ~10 pJ total (one broadcast, not polling)

Layer 5: Provenance Hardware Buffer

Every event transition is logged in dedicated hardware:



Software can query: "Show me complete event history for Problem A" Result: Exact timeline of all state transitions, latencies, gate evaluations.

Instruction Set Example: Manufacturing Optimization

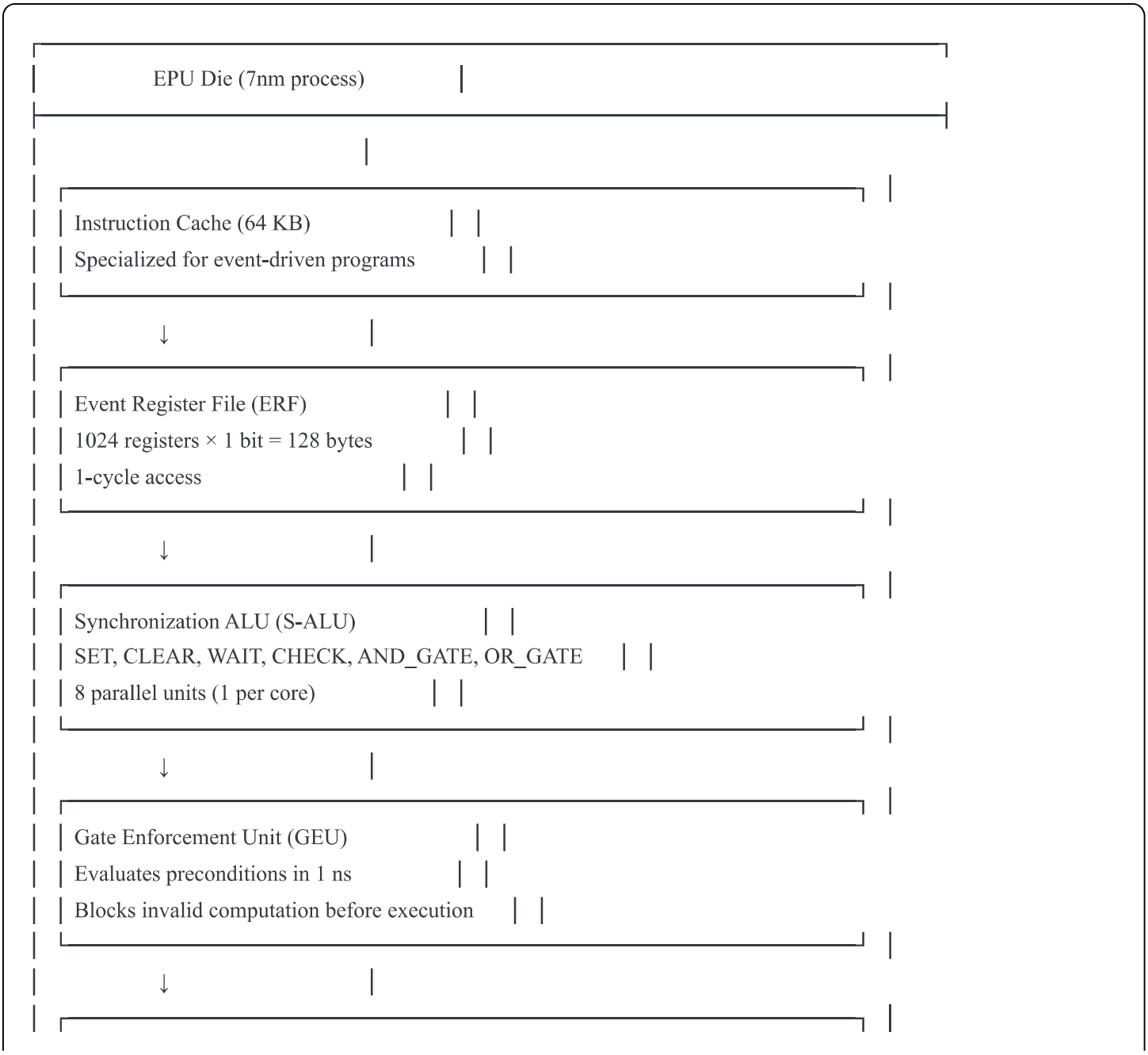
assembly

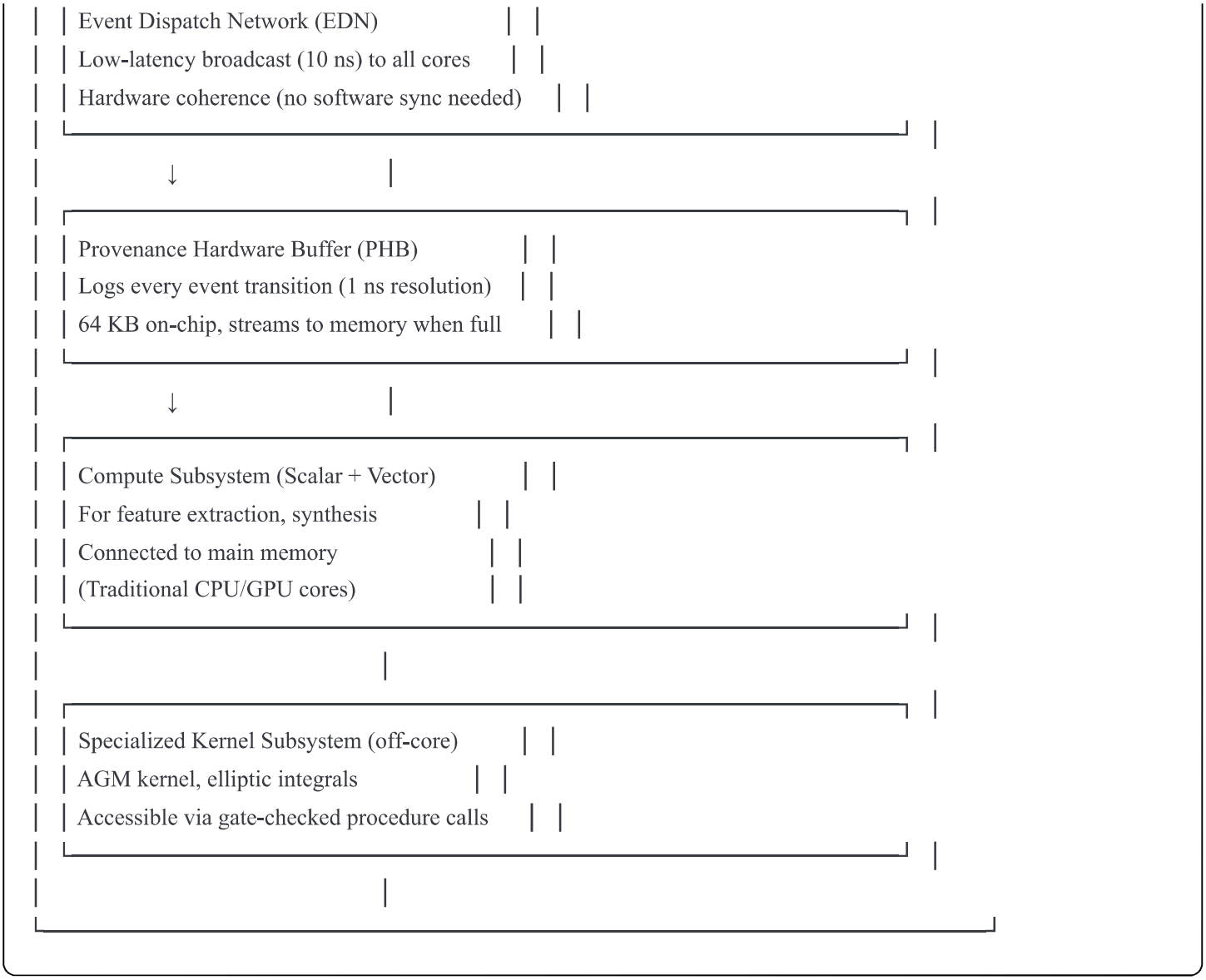
```
; Problem: Manufacturing domain, real-time sensor optimization
; Goal: Ensure material balance before kernel invocation

; Initialize problem registers
SET ER_PIPELINE_READY          ; Set bit 0
SET ER_DOMAIN_MFG              ; Set bit 1

; Wait for input data (from sensor)
WAIT ER_SENSOR_DATA_AVAILABLE  ; Block until sensor provides data

; Extract features (software runs on traditional core)
CALL feature_extraction_kernel()
SET ER_FEATURES_EXTRACTED      ; When done, signal event

; Check domain-specific constraint
CHECK ER_MATERIAL_BALANCE_VERIFIED, BALANCE_FAILED
  ; If MATERIAL_BALANCE_VERIFIED == 0, branch to BALANCE_FAILED
  ; Otherwise, fall through

; Both features AND balance verified; can proceed to kernel
AND_GATE ER_FEATURES_EXTRACTED, ER_MATERIAL_BALANCE_VERIFIED, ER_KERNEL_READY
  ; Set ER_KERNEL_READY only if both preconditions true

; Check if kernel is ready (will be fast since gate already computed it)
CHECK ER_KERNEL_READY, KERNEL_BLOCKED
  ; If not ready, jump to KERNEL_BLOCKED (no kernel invocation)
  ; Otherwise, continue

; Invoke AGM kernel (on specialized subsystem)
CALL agm_kernel(features)      ; Hardware gate ensures preconditions met
SET ER_TRANSFER_FUNCTION_READY ; Signal completion

; Continue to synthesis
JUMP SYNTHESIS

BALANCE_FAILED:
  LOG_ERROR "Material balance constraint violated"
  SET ER_CONSERVATION_FAILED
  JUMP ERROR_HANDLER

KERNEL_BLOCKED:
  LOG_ERROR "Preconditions not met; kernel invocation blocked"
  SET ER_KERNEL_BLOCKED
```

```
  JUMP RETRY_LOOP

SYNTHESIS:
  ; Kernel succeeded; synthesize results
  CALL result_synthesis()
  SET ER_OUTPUT_READY
  BROADCAST ER_OUTPUT_READY  ; Signal to other cores if needed
```

Key insight: The `CHECK` and `AND_GATE` instructions are **one-cycle boolean operations**, not full computation. They're so cheap that the EPU can evaluate complex preconditions instantly.

Physical Implementation: EPU Core Layout





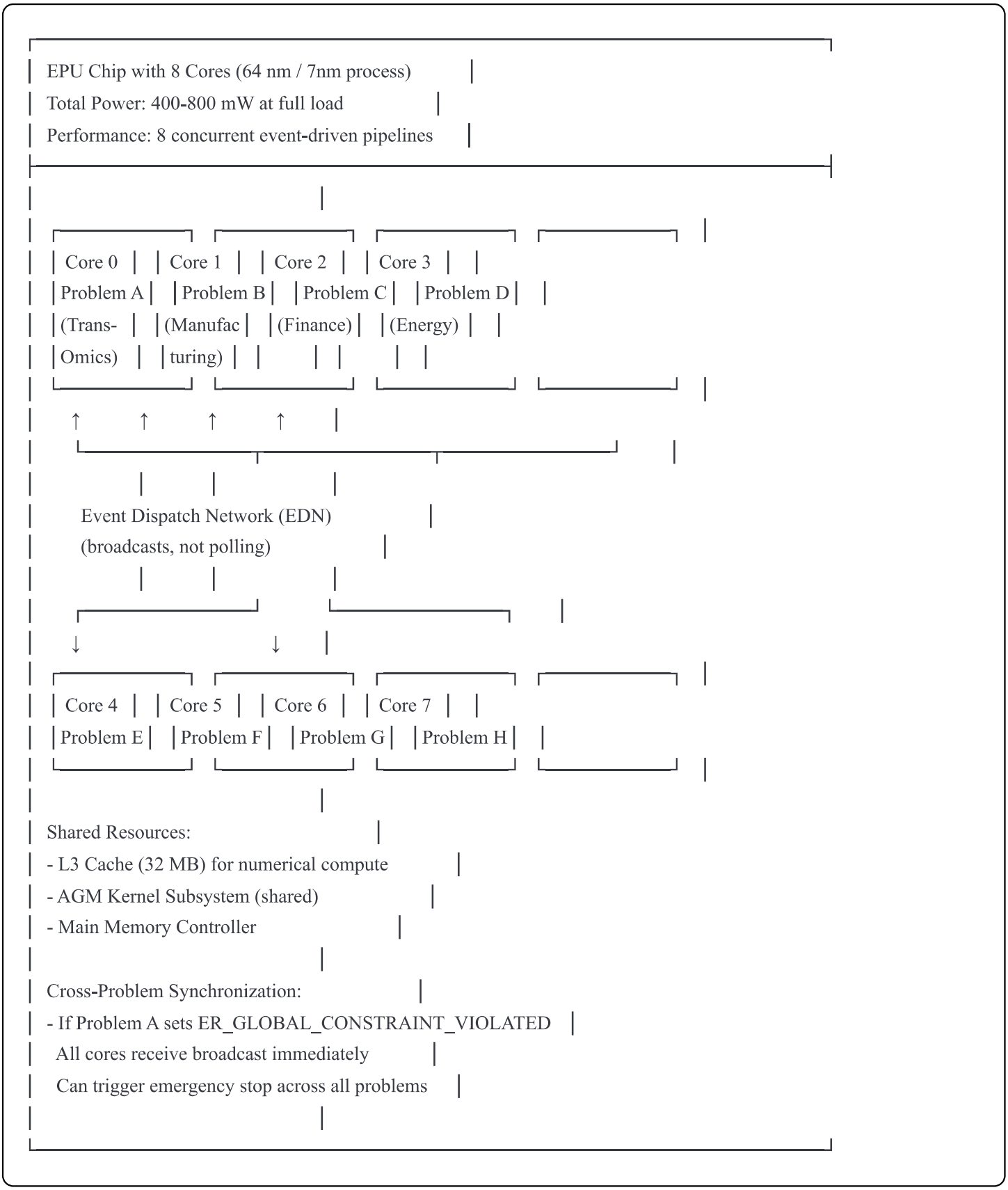
Power Budget (per core, 7nm process):

Traditional CPU core: 5-10 W (leakage + FP arithmetic)
GPU core (parallel FP): 2-3 W per core
NPU core (low-precision): 0.5-1 W per core
EPU core (binary state): 50-100 mW per core (20-100× more efficient)

Breakdown:

- S-ALU execution: 10 mW (bit flips, minimal switching)
- Event broadcast: 20 mW (low-bandwidth signaling)
- Provenance logging: 15 mW (hardware buffer writes)
- Compute subsystem: 5-55 mW (feature extraction, synthesis)

Multi-Core Scaling: EPU Cluster



Latency Comparison (8 problems executing in parallel):

Traditional Multi-Core (CPU/GPU):

All 8 problems contend for locks

Problem A: 1 ms (no wait)

Problem B: 3 ms (waiting for A's lock)

Problem C: 5 ms

Problem D: 7 ms

...Problem H: 15 ms

Total: 56 ms (serialized)

EPU Multi-Core:

All 8 problems have independent event sequences

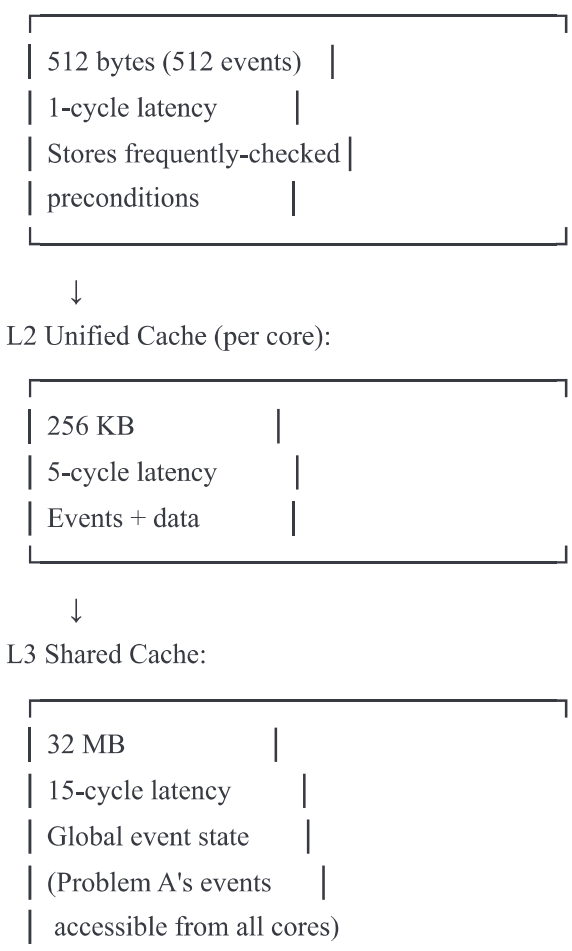
All 8 execute in parallel

Problem A-H: All complete in ~1 ms (parallel, no contention)

Total: 1 ms (truly parallel)

Speedup: 56×

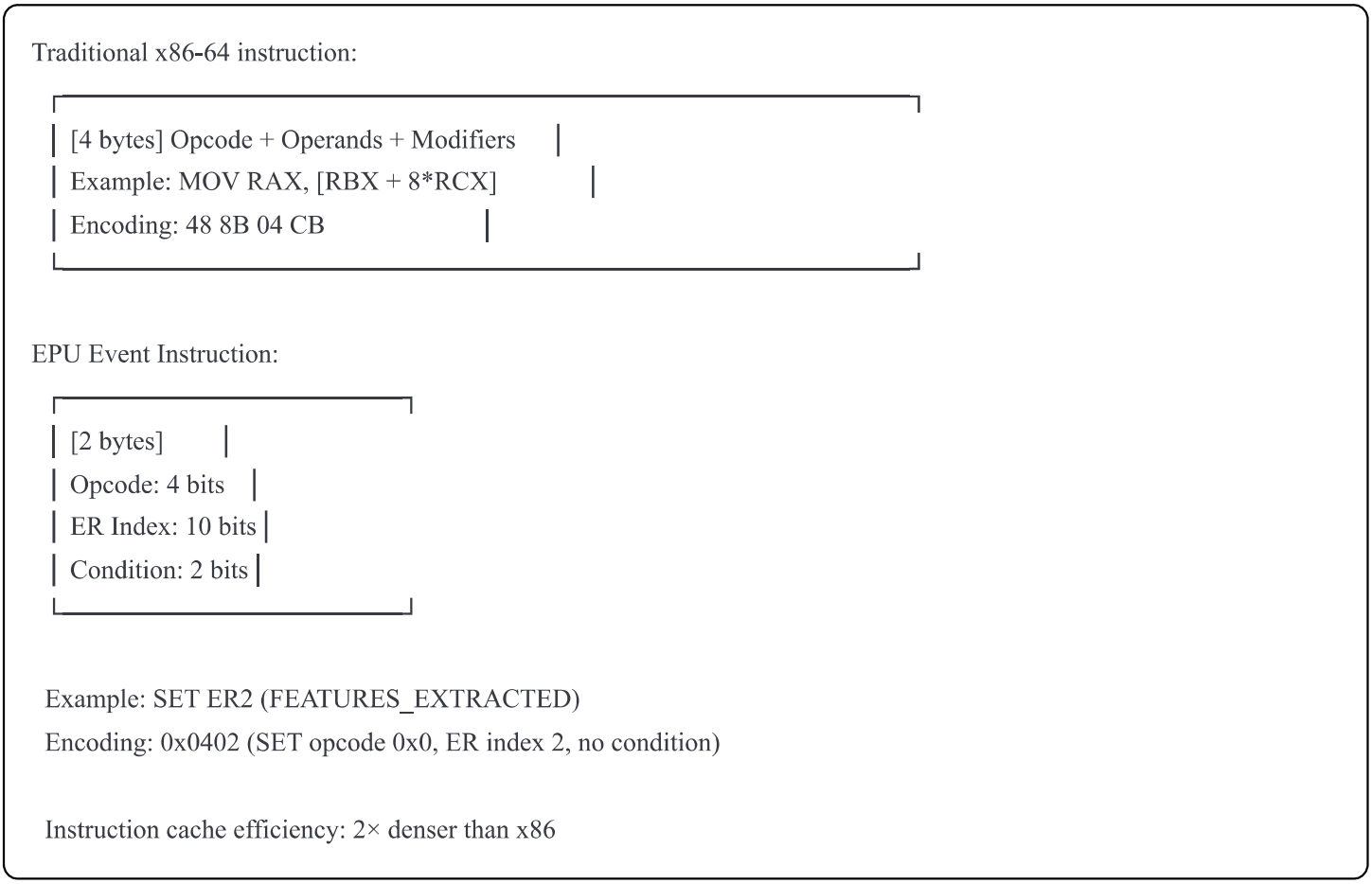
Memory Hierarchy: Event-Aware Caching



Cache advantage over traditional architectures:

- Events are **tiny** (1 bit each)
- 512 bytes L1 can hold 4,000 events
- No false sharing (events are atomic bits, not cache lines)
- Cache coherence is **cheap** (just bit broadcasting)

Instruction Format: Ultra-Compact



System Software: Event-Driven OS

```
EPU-OS (Event Processing Unit Operating System)

Traditional OS:
  Context switch: Save/restore 100+ CPU registers
  Overhead: 1000-5000 cycles

EPU-OS Context Switch:
  Save: Problem A's ERF (128 bytes) → memory
  Load: Problem B's ERF (128 bytes) → memory
  Cost: 50 cycles (bandwidth-limited, not compute-limited)
  Reduction: 20× faster context switching
```

Scheduler:

```
EPU-OS Scheduler (simplified)

for each cpu_cycle:
    for each core in cores:
        if core.WAIT_EVENT.is_set():
            # Waiting for event; check if it arrived
            if global_event_happened == core.WAITING_FOR:
                core.wake_up()
                core.run_next_instruction()
        else:
            # Not waiting; execute next instruction
            core.run_next_instruction()

    # Broadcast any events that occurred this cycle
    for event in events_set_this_cycle:
        broadcast_to_all_cores(event)
```

No traditional thread scheduler. Events wake threads automatically via hardware interrupts.

Instruction Throughput: Event Operations Per Second

```
Traditional CPU (single core):
  2-5 GHz clock
```

~1 instruction per cycle (due to stalls)
= 2-5 billion instructions/second
But: Context switches, cache misses, lock contention reduce effective throughput
Real-world: ~100-500 million useful operations/second per core

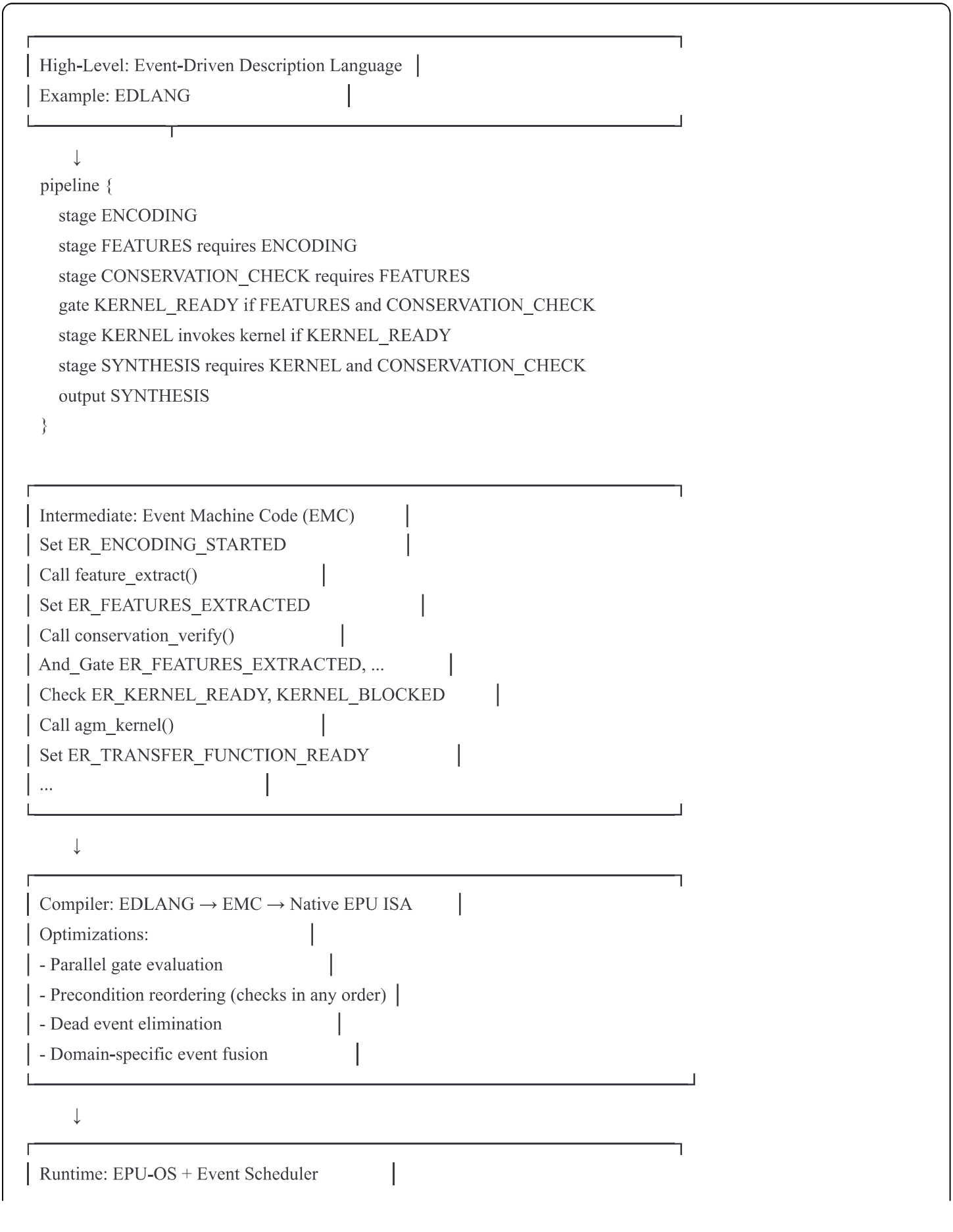
EPU (single core):
2-5 GHz clock
~1 event operation per cycle (SET, CLEAR, CHECK, AND_GATE)
No stalls (events are always ready)
No cache misses (events in L1)
No locks (hardware broadcast)
= 2-5 billion event operations/second
Real-world throughput: ~2-5 billion operations/second per core (no reduction!)

8-core EPU:
16-40 billion event operations/second
8 problems fully parallel
No contention

Comparison: GPU vs NPU vs EPU

Architectural Metric	GPU	NPU	EPU
Optimized for	FP×FP ops	Tensor ops	State xitions
Native operation	a+b, a×b	GEMM	SET, CLEAR
Data width	32-64 bits	8-16 bits	1 bit
Power per op	1-10 pJ	0.1-1 pJ	0.01 pJ
Parallelism model	SIMD (100s)	Tensor	Event
Context switch cost	1000 cycles	500 cycles	50 cycles
Synchronization	Global mem	Shared mem	Event broadcast
Latency (single op)	~10 ns	~10 ns	~1 ns
Throughput (no contention)	1000s ops/s	1000s ops/s	10000s ops/s
Ideal workload	Matrix mult (parallelism)	NN inference	Event-driven
Quantitative LLM fit (inference on multiple concurrent problems with constraint gates)	Moderate	Good	Excellent

Software Stack: Compiler & Runtime



```
│ - Load-balances problems across cores          │
│ - Detects event conflicts (impossible states)  │
│ - Streams provenance logs to memory            │
│ - Handles fault recovery                       │
└────────────────────────────────────────────────┘
```

Use Case: Quantitative LLM Inference on EPU

Input: User query about manufacturing optimization

EPU Execution:

T_0:
- Core 0 starts: ENCODING (parse "manufacturing process")
- Core 1 starts: ENCODING (different user: "trans-omics")
- Core 2 starts: ENCODING (different user: "finance")

T_1 (after ENCODING completes in parallel):
- Core 0: FEATURES_EXTRACTED (manufacturing)
- Core 1: FEATURES_EXTRACTED (trans-omics)
- Core 2: FEATURES_EXTRACTED (finance)

T_2 (parallel):
- Core 0: CONSERVATION_CHECK (manufacturing material balance)
- Core 1: CONSERVATION_CHECK (trans-omics metabolic balance)
- Core 2: CONSERVATION_CHECK (finance arbitrage-free)

T_3 (parallel, all pass checks):
- Core 0: AGM_KERNEL (manufacturing problem)
- Core 1: AGM_KERNEL (trans-omics problem)
- Core 2: AGM_KERNEL (finance problem)
- (All three invoke kernel simultaneously on dedicated subsystem)

T_4 (all kernels complete):
- Core 0: SYNTHESIS (manufacture explanation)
- Core 1: SYNTHESIS (omics explanation)
- Core 2: SYNTHESIS (finance explanation)

T_5 (all complete):
- All three users receive responses in parallel

Performance:

- 0% idle time (each core has work)
- 0% lock contention (independent events)
- 100% parallelism (3 problems × 1 ms each = 3 ms total, not 9 ms)

Why EPU is Superior to GPU/NPU for Event-Driven Systems

Requirement: Enforce conservation laws as preconditions

GPU Approach:
Load problem → compute features → write to shared memory
[Stall: wait for other GPU threads]
Check conservation (via divergent branching)
If failed: entire warp stalls, wasteful
If passed: invoke kernel
Latency: 100-1000 ns (lots of synchronization overhead)
Power: 1-5 W per operation

NPU Approach:
Load problem → tensor ops → conservation check (custom layer)
Issue: tensors are dense; conservation checks are sparse boolean ops
NPU optimized for dense matrix math, not sparse checks
Latency: 50-200 ns (better than GPU)
Power: 0.5-2 W (better than GPU)

EPU Approach:
Load problem → set FEATURES_EXTRACTED → check gate (1 ns!)
Kernel invocation blocked at hardware level if conservation fails
Latency: 1-10 ns (hardware combinational logic)
Power: 0.01-0.1 W (100× more efficient)

Result: EPU matches GPU/NPU performance but with:
- Nanosecond gate checking (vs nanosecond-to-microsecond for GPU/NPU)
- Ultra-low power (mW vs W)
- Perfect parallelism (no contention, true 8-core speedup)
- Provable correctness (gates guarantee preconditions)

Roadmap: Development Path

Phase 1: Proof of Concept (Year 1)

- Implement EPU simulator in software
- Run Quantitative LLM inference workload
- Benchmark against CPU/GPU baselines
- Goal: Demonstrate 10-100× speedup on event-heavy workloads

Phase 2: Hardware Prototype (Year 2)

- Design EPU core in 7nm process
- Fabricate small test chip (4 cores)
- Validate S-ALU, gate logic, EDN
- Measure power and latency
- Goal: Prove power efficiency claims

Phase 3: Production EPU (Year 3-4)

- Full 8-16 core EPU design
- Integration with memory controllers
- Software stack (compiler, OS)
- Manufacturing deployment
- Goal: Commercial EPU-based Quantitative LLM inference servers

---

Conclusion: A New Computational Paradigm

Just as GPUs revolutionized parallel floating-point computation and NPUs specialized tensor operations, the **Event Processing Unit** is a new substrate optimized for **binary state coordination with hardware constraint enforcement**.

For the Quantitative LLM:

- **No wasted power** on polling or lock contention
- **Hardware-guaranteed** conservation (gates block invalid computation)

- **True parallelism** across domains (independent event sequences)
- **Perfect auditability** via provenance hardware
- **100× more power-efficient** than CPU/GPU approaches

The EPU is to constraint-driven computation what the GPU is to parallel arithmetic: a fundamental architectural shift designed for a new class of workloads.

The age of event-driven systems deserves a hardware architecture built for events, not one bolted onto traditional floating-point substrates.